29/03/2026

optimal sol^n of union of two sorted Arrays

→ figure out size of first array

figure out size of second array

i pointer to iterate elements of first array, j pointer to iterate elements of second array. result vector to store union array.

we will compare till

```
i < n1  && j < n2
    if ( a[i] <= b[j] )    // a has smaller element
        if (result.size() == 0 ||
            result.back() != a[i])
        {
            result.push_back(a[i]);
        }
        i++
    else
        // b has smaller element
        if (result.size() == 0 ||
            result.back() != b[j])
        {
            result.push_back(b[j]);
        }
        j++
```

while comparing suppose any one array will be exhausted.

```
while (j<n2){
    // same thing for second array
}

while (i<n1){
    //same thing for first array
}

return result
```

TC : O(n1 + n2)
SC : O(n1+n2) for returing the result vector.

Missing Element

Bruteforce approch :

N = arr.size() + 1

Iterate for 1 to N. maintain flag variable which was intialised to false.

```
Iterate 0 to arr.size()
if arr[j] == i
    ↳ flag = true
```

At the end of first for loop suppose flag is false means arr[i] != i & thus return its index.

TC => $O(N^2)$

Better approch

sort the array

Iterate over the length of array & search if i+1 != arr[i] then return i+1.

TC => $O(n\log n) + O(n)$

$\therefore \sim O(n\log n)$

Best / optimal Approch

N = arr.size() + 1

take sum from 1 to N $\left[\because n*\frac{(n+1)}{2}\right]$

find sum of all the elements of array.

After subtraction we will get the missing number.